

NovelForge

小说工坊

—— 本地优先、零 AI 依赖的中文长篇小说创作平台 ——

项 目	内 容
项目名称	NovelForge (小说工坊)
版本号	v1.0 · 阶段 6 进行中
文档类型	详细项目计划书
编写日期	2026-09-04
技术栈	Tauri 2 · Rust (后端, SQLite 内嵌) + React 19 + TypeScript (前端)
部署形态	纯本地 Windows 桌面软件 (NSIS 安装包, 无网络依赖)
核心设计原则	没有 AI, 软件的价值 = 确定性计算 + 信息管理

第一章 项目概述

1.1 项目背景

目前市面上的中文小说写作工具可分为三类：一是以《墨刀》《有道云》为代表的云笔记类工具，胜在方便但缺少小说特有的章节/分卷结构，更缺少导入、导出、备份等出版级能力；二是以《笔趣阁》《起点作家助手》为代表的平台自带编辑器，其数据归平台所有、作者无法迁移；三是以Manuskript、novelWriter、bibisco 为代表的开源桌面软件，功能完整但不贴近中文写作习惯（中文标题识别、中文分词友好度、GB18030 编码兼容、卷章结构匹配网文连载模式）。

NovelForge 正是针对以上缺口诞生：以 Rust 内嵌 SQLite 为核心数据层，保证单章独立存储、崩溃可恢复；以 React 19 构建三栏式（目录树 + 编辑器 + 信息面板）中文写作界面；严格遵守本地优先原则，不接入任何 AI 服务，所有功能依靠确定性计算与信息管理的实现。

1.2 项目定位与目标用户

- 核心用户：网文作者（起点/番茄/飞卢平台长篇连载作者，50 万字以上量级）
- 次级用户：传统文学小说作者、编剧、剧本创作者
- 用户核心痛点：丢稿怕、防盗 TXT 导入乱、章节管理混乱、每天日更没动力、人物登场乱忘、平台机审卡敏感词

1.3 差异化价值主张

NovelForge 与同类产品相比的四大差异化：

- 极致离线安全：数据库 SQLite 内嵌；项目文件 = 一个可拷贝目录；无账户、无云、无分析上传；30 分钟自动 ZIP 备份 + 章节级 50 版本回滚，从根子上解决丢稿焦虑。
- 中文导入天花板：实测《捡漏》26MB GB18030 文件 4553 章秒级拆分；支持防盗填充自动检测与剔除（同段占位内容重复 456 次时精准识别）、TXT/DOCX 双格式、用户可视预览+边界调整。
- 零噪声信息管理：放弃阶段 5 中「自动识别人物」的识别流程（真实网文上人物候选高达 2374 条，确认流程不可扩展），改道为「作者手动建卡 + 引擎精确匹配统计」的模式——名字+别名计数 100% 精确，零确认负担。
- 作者感知强的日常：码字统计仪表盘（今日字数、连续写作天数、近 60 天柱状图、最佳日、活跃天数），精准戳中网文作者「日更 4000、全勤奖就是钱」的刚需。

第二章 功能范围与阶段规划

2.1 阶段路线图（已完成 5/6）

阶段	主题	核心交付	状态
阶段 1	项目骨架 + 编辑器	新建/打开项目、目录树、三栏布局、自动保存 500ms	已完成
阶段 2	卷章管理	新建/重命名/移动/删除/分卷、状态标记、字数统计、批量操作	已完成
阶段 3	导入系统	TXT / DOCX 导入、GB18030 兼容、防盗填充检测、章节边界调整 UI	已完成
阶段 4	导出 + 搜索 + 备份	TXT/DOCX/MD 三格式导出、全文检索（含片段高亮）	已完成
阶段 5	实体识别（已改道）	自动人物/地点识别已废弃；改道手动建卡+精确匹配系统	已完成
阶段 6	码字统计 + 手动人物卡	日码字统计与仪表盘、连续打卡、人物/地点手动 CRUD	进行中
阶段 7	校对与发布	敏感词扫描、错别字检查、重复口头禅检测、完本进度待启动	待启动

2.2 已实现功能清单（截至阶段 5）

模块	已实现能力	代码位置
项目骨架	create_project / open_project / close_project / 最近项目 20 条	9 commands
编辑器	行内标题编辑、500ms 防抖 + 5min 快照 + Ctrl+S 快照、Tab 全角缩进、实时字数/段落/阅读时长	
卷章管理	create/rename/move/delete/set_status 全部事务化；跨卷移动 SQL 事务重排	
导入系统	TXT + DOCX；7 条识别规则 + 中文边界；重复内容四重指纹；预览确认 filex · 84 tests pass	
导出系统	TXT / DOCX / MD；范围（全书/卷/单章）；DOCX 含自动目录 3 writers	
全文搜索	SQL LIKE + 正则；前后文片段 + 命中数 badge	1 command
备份系统	30 分钟自动 ZIP + 手动备份 + 列表 + 恢复前自动冲刷	backup.rs
版本快照	chapter_versions 每章最多 50 条；恢复前自动备份当前版	InfoPanel 快照卡
主题系统	深色 / 浅色 / 护眼三主题循环切换 + 字体字号行距	SettingsModal

2.3 阶段 6 本轮交付（进行中）

A. 码字统计仪表盘（新增）

- writing_daily 表 (YYYY-MM-DD 主键)：保存章节时正增长记账（删除不扣减）
- 今日 +N 字数实时刷新 (StatusBar 右侧同步显示)
- 连续写作天数 streak（今天未写则从昨天起算）
- 活跃天数、单日最高字数 + 近 60 天柱状图
- 全书当前总字数 / 章节数一览

B. 人物 / 地点卡改造（替换阶段 5 鸡肋识别）

- DB 迁移 v3：清除 `status!=1` 的候选/忽略实体；废弃 `events / timeline_markers / analysis_cache` 三表
- 新增 `matching.rs` 精确匹配引擎：名字+别名构建正则交替式，最长优先不重叠计数
- 人物 CRUD：手动新建、重命名、别名管理、角色定位、备注、删除
- 人物统计：总出场、出场章节、首现章、最近章、断档预警、全书热度条（`per_chapter` 对齐）
- 地点 CRUD + 同样精确匹配统计
- 本章出场卡（`InfoPanel` 替换分析卡）：当前章节中已建卡的人物/地点及出现次数

第三章 技术架构设计

3.1 整体架构图（文字版）

前端（React 19 + TypeScript + Vite + Zustand）通过 Tauri 2 IPC 通道调用后端 commands；后端（Rust + Tauri 2）分为 commands 编排层、db 迁移/连接层、import/analysis/text 纯函数层；数据层由内嵌 SQLite（WAL + 外键 + NORMAL 同步级别）承载，<项目目录>/database/novel.db 为单项目库，%APPDATA%/NovelForge/app.db 为全局库（最近项目）。

3.2 技术栈选型与理由

技术	角色	选型理由
Tauri 2	桌面壳	Electron 的 1/10 体积（3.5MB vs 120MB+）；Rust 内存安全；原生菜单/窗口/文件
Rust + rusqlite (bundled)	数据层	SQLite 编译进二进制，用户无需安装任何数据库；事务保证保存/备份/导入原子性
React 19 + TypeScript 5.6	UI	生态成熟；tsc --noEmit 构建前类型检查，降低 runtime 错误
Zustand 5	状态管理	相比 Redux 体积小、样板少；两个 store 分工：appStore（项目级）+ editorStore
regex 1.13	导入识别 / 精确匹配	正则生态最成熟，支持 Unicode 文本处理，不回溯性能高
quick-xml + encoding_rs	DOCX 导入 / 导出	DOCX 的 word/document.xml 解析；TXT 的 GB18030 解码（实测 26MB 文件）
zip 8.6 (deflate)	备份压缩 / DOCX 功能裁剪	干净，仅启用放气压缩，无多余依赖
NSIS	Windows 安装包	Tauri 自带 bundle；已实测发布 NovelForge_0.1.0_x64-setup.exe

3.3 数据库 Schema 全景（v3 迁移后）

表名	用途	关键字段
project_info	单行表	项目名/作者/简介/创建更新时间/schema_version
volumes	卷	标题/简介/排序/更新时间
chapters	章节（核心）	所属卷/标题/正文/字数/字数字符数/状态/梗概/笔记/内容指纹（FNV-1a）
chapter_versions	章节历史快照	每章最多 50 条；自动/手动/恢复前备份三类型
characters	人物卡	名字 UNIQUE/别名 JSON/角色定位/备注（全部手动，无 status 列）
locations	地点卡	名字 UNIQUE/备注
character_mentions	人物 × 章节	精确匹配计数，Upsert 增量更新；作为热度条/首末章/断档统计数据源
location_mentions	地点 × 章节	同理（精确匹配）
writing_daily	码字统计	日期主键/当日净增正字数/保存次数；统计仪表盘唯一数据源
settings	KV 设置	编辑器偏好等（阶段 4 已预留）

迁移策略：PRAGMA user_version 顺序迁移，新增功能仅追加 (version, SQL)
对，不修改历史；新版本软件打开旧项目自动补齐。

3.4 核心数据流

自动保存流

用户输入 → editorStore.setContent (dirty=true) → useAutoSave 500ms 防抖 → save(false)
走 tauri IPC → save_chapter 事务：更新正文 + 字数 + 指纹 + 写作日记账；若
snapshot=true (5 分钟周期或 Ctrl+S)，还额外 INSERT chapter_versions + 清理超出 50
条的旧版；前端回调刷新字数、今日字数、树统计。

导入流

importAnalyzeFile 读取 TXT/DOCX → 编码检测 (encoding_rs) → 7 规则切章 →
四重指纹去重 (完全相同/内容指纹/首末段指纹/整章 hash) → 返回 ImportAnalysis (章节预览 +
confidence + duplicate_of 来源) → 用户调整 excluded/discarded → importConfirm
事务一次性落库 chapters/volumes 并返回 ImportResult。

精确匹配计数流

作者在 AnalysisModal 新建人物卡 (名字+别名) → matching.rs
构建「名字|别名1|别名2...」正则 (最长优先、元字符转义) → 遍历全书章节统计提及数 → 写入
character_mentions (Upsert) → get_characters 聚合出首末章/总出场/章节数/断档。

第四章 阶段 6 详细实施方案

4.1 后端改造清单 (Rust)

#	模块	改动要点	规模
1	migrations.rs	追加 writing_daily 表、清候选数据、DROP 派生三表	新增 ~40 行
2	matching.rs (新增)	精确匹配正则计数 + 8 单测 (别名合并/长词优先/元字符转义)	90 行 · 8 tests
3	models.rs 重构	CharacterProfile/CharacterHeat/LocationProfile 去掉 status; 新增 ChapterPresenceView	改 ~200 行
4	chapter.rs save_chapter	保存时计算 (new_words - old_words).max(0), UPSERT writing_daily 当日; 返回 today_words	改 ~20 行
5	analysis.rs 重构	删 analyze_project / set_character_status / set_location_status; 新增 chapter_analysis / get	改 ~300 行
6	lib.rs 命令注册	移除旧命令, 注册新命令	改 ~20 行
7	db/mod.rs 注释	修正 V3 以后结构说明	改 ~5 行

4.2 前端改造清单 (React + TS)

#	文件	改动要点	规模
1	api/index.ts	去 analyzeProject / setCharacterStatus / setLocationStatus; 新增 getBookAnalysis / getChapter	改 ~60 行
2	types/models.ts	同步 models.rs 类型 (删 status; 新增 DailyWords、WritingStats、ChapterPresenceView)	改 ~80 行
3	editorStore.save	返回 today_words 同步写入状态	改 ~5 行
4	AnalysisModal.tsx	重构为「人物/地点」两 Tab; 删候选/忽略/重新分析 UI; 重写成纯手动 CRUD + 统计展示 + 热	改 ~150 行
5	StatsModal.tsx (新增)	新增 1 个统计仪表盘: 今日/连击/活跃/最佳日 + 60 天柱状图 (新增 CSS 绘制, 无 echarts 依赖)	新增 ~60 行
6	TopBar.tsx	新增统计入口 (柱状图图标); 旧的人物库按钮保留 (指向 AnalysisModal)	改 ~10 行
7	StatusBar.tsx	右侧显示「今日 +XXXX 字」 (从 editorStore.today_words 读取)	改 ~10 行
8	InfoPanel.tsx	「本章分析卡」→「本章出场卡」, 显示已建卡人物/地点出现次数; 去掉时间线/事件	改 ~10 行
9	ProjectView.tsx	删除打开项目时自动 analyze_project 调用 (该命令已移除)	删 ~10 行
10	icons.tsx	新增 IconChart 柱状图 icon	加 ~15 行
11	global.css	StatsModal/出场卡相关样式 (.stats-hero / .stats-bar-root / .presence-chip)	加 ~80 行

4.3 测试计划

Rust 单元测试

- matching.rs: 8 个已写 (见上文)
- db/migrations: V1→V3 升级 (空库新建 + 含候选脏数据的 V2 库升级, 检查候选被清、writing_daily 表存在)
- analysis.rs rebuild_mentions: 建卡后已知输入的章节内容, 验证计数精确

- chapter.rs save_chapter: 正增长计入 writing_daily, 零增长与负增长不扣减

前端构建

- tsc --noEmit (vite build 前执行)
- cargo test (预期 ≥ 84 passed)

第五章 UI / UX 设计概述

5.1 设计风格延续

沿用已确认的「简约高级 + 纸墨编辑风」主题：背景 #FAF8F4（米纸）、边框 #E2DCCC（浅灰线）、主色 #3D5A80（靛蓝）、点缀色 #A66A2E（青铜）；字体为系统中文（微软雅黑/PingFang）回退机制，编辑器支持字体/字号/行距独立设置，内置 3 主题循环（深色/浅色/护眼）。

5.2 阶段 6 新增界面

界面	描述	位置
码字统计弹窗 (StatsModal)	仪表盘布局：顶部四张英雄卡（今日 / 连击 / 活跃 / 最佳日）800x300px 无柱状图（CSS div 绘制，无选中/忽略分组；直接是单列表（按总出场降序）；每行名字 300px + badge + 别名 chips + 统计行 + 地点卡 Tab (AnalysisModal) 同人物卡简化版；无别名/定位，只有名字、备注、统计、删除 300 Modal	中部 600px 无柱状图（CSS div 绘制，无选中/忽略分组；直接是单列表（按总出场降序）；每行名字 300px + badge + 别名 chips + 统计行 + 地点卡 Tab (AnalysisModal) 同人物卡简化版；无别名/定位，只有名字、备注、统计、删除 300 Modal
本章出场卡 (InfoPanel)	两个 chips 区域：人物 chips（「金锋 ×47」）+ 地点 chips 右侧点击可跳转到对应卡编辑（预留）	右侧点击可跳转到对应卡编辑（预留）
StatusBar 今日字数	在「全书 XXX 字」前增加「今日 +1280 字」青铜色 badge，底部有后平滑滚动数字	底部有后平滑滚动数字
TopBar 统计入口	新增柱状图 icon 按钮 (IconChart)，位于人物库旁，点击打开 StatsModal	顶部 StatsModal

5.3 交互原则

- 所有破坏性操作（删除人物/恢复历史版本/清空快照）一律二次确认
- 保存反馈：保存中呼吸灯、保存完成 0.8s 绿色闪光、字数滚动动画
- 快捷键一致：Ctrl+N 新章、Ctrl+F 搜索、Ctrl+S 保存+快照、Tab 全角缩进
- 错误友好：所有后端 command 抛错前端以 Toast 展示，不抛弹框打断写作

第六章 质量保障与测试策略

6.1 分层测试金字塔

- 单元层 (Rust)：matching.rs 精确匹配 8 项；导入 detector.rs 7 规则 + 重复检测；text.rs 字数统计/FNV 指纹
- 命令层 (Rust + SQLite)：migrations.rs 顺序迁移；chapter.rs 保存事务与快照裁剪；backup.rs ZIP 压缩解压往返
- 类型层 (TypeScript)：前端所有 API 返回、Model 类型与 Rust serde camelCase 完全对齐；tsc --noEmit 在构建前强制通过
- 集成层 (构建验证)：npm run build (tsc + vite) → cargo test → tauri build (NSIS 安装包) 三步 CI

- 手工冒烟：《捡漏》26MB 导入 + 保存一章 + 建 5 人物卡 + 导出 DOCX → 检查 mentions 计数、字数累计、ZIP 备份

6.2 性能指标（验收阈值）

场景	SLA
项目打开（空项目）	< 300 ms
项目打开（3k 章）	< 1.5 s
保存一章 4k 字	< 100 ms（无快照）， < 200 ms（含快照）
《捡漏》26MB 导入预览	< 5 s
导入确认落库（4500 章）	< 15 s
精确匹配全量重建（3k章 × 50 人物）	< 30 s（后台异步，不阻塞 UI）
ZIP 备份（3k 章项目）	< 5 s
安装包体积	< 10 MB（实测 3.5 MB，达标）

6.3 错误处理

后端统一 `AppError (thiserror 2) : Msg` 用户友好文本 + `Rusqlite/Sql`
原始错误入日志；前端 `api/client.ts` 所有 `invoke` 以 `Promise.reject` 抛出并 `Toast`
展示；导入/备份等长操作用互斥状态避免重入（`analyzing/saving/restoring` 布尔）。

第七章 项目里程碑与验收标准

里程碑	验收标准	计划时间
M1 · DB 迁移 v3 + matching 迁移执行 + 8 matching 测试全绿；旧库中 status!=1 的人名被正确清理		阶段 5 第 1 天
M2 · 码字统计后端	save_chapter 返回 today_words; get_writing_stats 输出段字量全部正确；Rust 测试覆盖正	阶段 5 第 2 天
M3 · 人物卡后端改造	精确匹配 mentions 重建+增量；add/update/delete/help 全链路 2-3 天	阶段 6 第 1 天
M4 · AnalysisModal 重构 人物/地点两 Tab 纯手动 CRUD；候选/忽略/重新分析 UI 除移除陈;3 断档预警、热度条显示		阶段 6 第 2 天
M5 · StatsModal + StatusBar 仪表盘四字数 60 天柱状图（无 ECharts 依赖）；状态栏今日字数统计保存刷新		阶段 6 第 3 天
M6 · InfoPanel 出场卡 + 章节出场卡替换分析卡；ProjectView 删除自动 analyze_project 阶段 6 第 4 天		阶段 6 第 4 天
M7 · 发布	cargo test ≥ 84 passed; tsc --noEmit; tauri build 生成新 1.0 安装包	阶段 6 第 5 天

第八章 风险与应对

编号	风险描述	等级	应对措施
R1	精确匹配人名重叠（如「苏婉」误匹配「苏婉清」）	低	在 Rust 中已按最长优先+不重叠计数解决；单元测试覆盖；极端情况作
R2	migration v3 在用户大项目上耗时过长	低	所有 DELETE 走索引（status/外键）；SQLite 在 10 万行以下为 <1s；且
R3	码字统计负值争议（删改后净增为低，要不要算？只计正增长 (.max(0)) ——作者删改视为正常迭代，不惩罚今日码	低	
R4	NSIS 安装包在升级时覆盖用户数据库	低	数据库在 <项目目录>/database/ 下而非 AppData；升级只替换安装目录
R5	60 天柱状图 0 数据时 UX 空洞	低	StatsModal 空态引导：「尚未有码字记录，开始写下第一章吧」；今日第

第九章 团队与协作方式

本项目当前为单人核心开发 + AI 辅助开发（TRAE Workspace）模式。协作方式：TraeWork 会话作为单一事实来源（SSOT）；每次功能迭代开始前先 review 阶段进度截图确认需求 → 细分任务清单（TodoWrite 工具）→ 先测试后代码 → Rust + TS 双重检查 → 生成安装包；阶段验收通过 TraeWork 分享链接 + 安装包交付。

第十章 预算估算

费用项	金额（元/年）	备注
开发人力（单人 · 15 天/阶段 × 6 阶段）	6,000	参考单价 1000 元/天；实际自开发为零直接成本
云基础设施	0	纯本地软件，零服务器、零 API、零 CDN
AI 辅助开发订阅（TRAE）	0（自用） / ~300/月	TRAE Workspace 专业版
代码签名证书（可选，免 SmartScreen）	~1,500/年	EV 代码签名；当前阶段暂不需要

费用项	金额（元/年）	备注
分发渠道（自建官网/作者分享）	~200/年	域名+静态空间；或纯网盘分发，成本 0
合计（自开发 + 暂不签名）	≈ 0 ~ 500	第一年运行成本极低

第十一章 结语

NovelForge 的核心信念是：好的写作软件应该像一张结实的书桌——不喧宾夺主、不丢稿、不添麻烦，只在作者每天打开它的那几个小时里，可靠地替他管好书、记好字、算好数。

阶段 5 的改道（砍掉鸡肋识别）让我们更坚定地回到了这条主线：不做需要语义理解的花哨功能，只把作者真正每天要面对的痛点——导入脏乱、字数记录、人物记混、怕丢稿、全勤奖——用纯粹的确定性计算一件件解决。阶段 6 完成后，NovelForge 将从「能写小说的编辑器」正式升级为「作者每天愿意打开的写作工具」。